

本节内容

无符号整数的 乘法运算原理

本节总览

无符号整数的 乘法运算原理

十进制的手算乘法 → 二进制数的手算乘法

无符号数的乘法电路结构、运行原理

电路结构中，各个部件的作用

手算乘法（十进制）

你怎么这个亚子

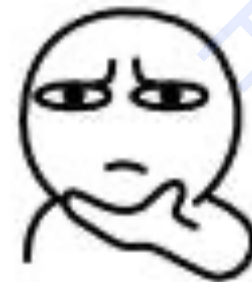


$$\begin{array}{r} 985 \\ \times 211 \\ \hline 985 \\ 985 \\ 1970 \\ \hline 207835 \end{array}$$

为什么要
“逐位相乘，错
位相加”？



$$\begin{array}{r} 985 \\ \times 211 \\ \hline 985 \\ 9850 \\ 19700 \\ \hline 207835 \end{array}$$



(被乘数) 985

(乘数) $211 = 2 \times 10^2 + 1 \times 10^1 + 1 \times 10^0$

$$985 \times 211 = (985 \times 2 \times 10^2) + (985 \times 1 \times 10^1) + (985 \times 1 \times 10^0)$$

手算无符号整数乘法（二进制）



结论：无符号整数（二进制）的手算方法与十进制类似——“逐位相乘，错位相加”

$$\begin{array}{r} 1101 \\ \times 1011 \\ \hline 1101 \\ 1101 \\ 0000 \\ 1101 \\ \hline 10001111 \end{array}$$



$$\begin{array}{r} 1101 \\ \times 1011 \\ \hline 1101 \\ 11010 \\ 000000 \\ 1101000 \\ \hline 10001111 \end{array}$$

用十进制验算：
 $13 * 11 = 143$

(被乘数) 1101

(乘数) $1011 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$

$$1101 \times 1011 = (1101 \times 1 \times 2^3) + (1101 \times 0 \times 2^2) + (1101 \times 1 \times 2^1) + (1101 \times 1 \times 2^0)$$

手算无符号整数乘法（二进制）

$$\begin{array}{r} 1101 \\ \times 1011 \\ \hline 1101 \\ 1101 \\ 0000 \\ 1101 \\ \hline 10001111 \end{array}$$



$$\begin{array}{r} 1101 \\ \times 1011 \\ \hline 0000 \quad \text{部分积 } P_0 \\ + 1101 \\ \hline 1101 \quad \text{部分积 } P_1 \\ + 1101 \\ \hline 100111 \quad \text{部分积 } P_2 \\ + 0000 \\ \hline 100111 \quad \text{部分积 } P_3 \\ + 1101 \\ \hline 10001111 \quad \text{最终乘积 } P_4 \end{array}$$

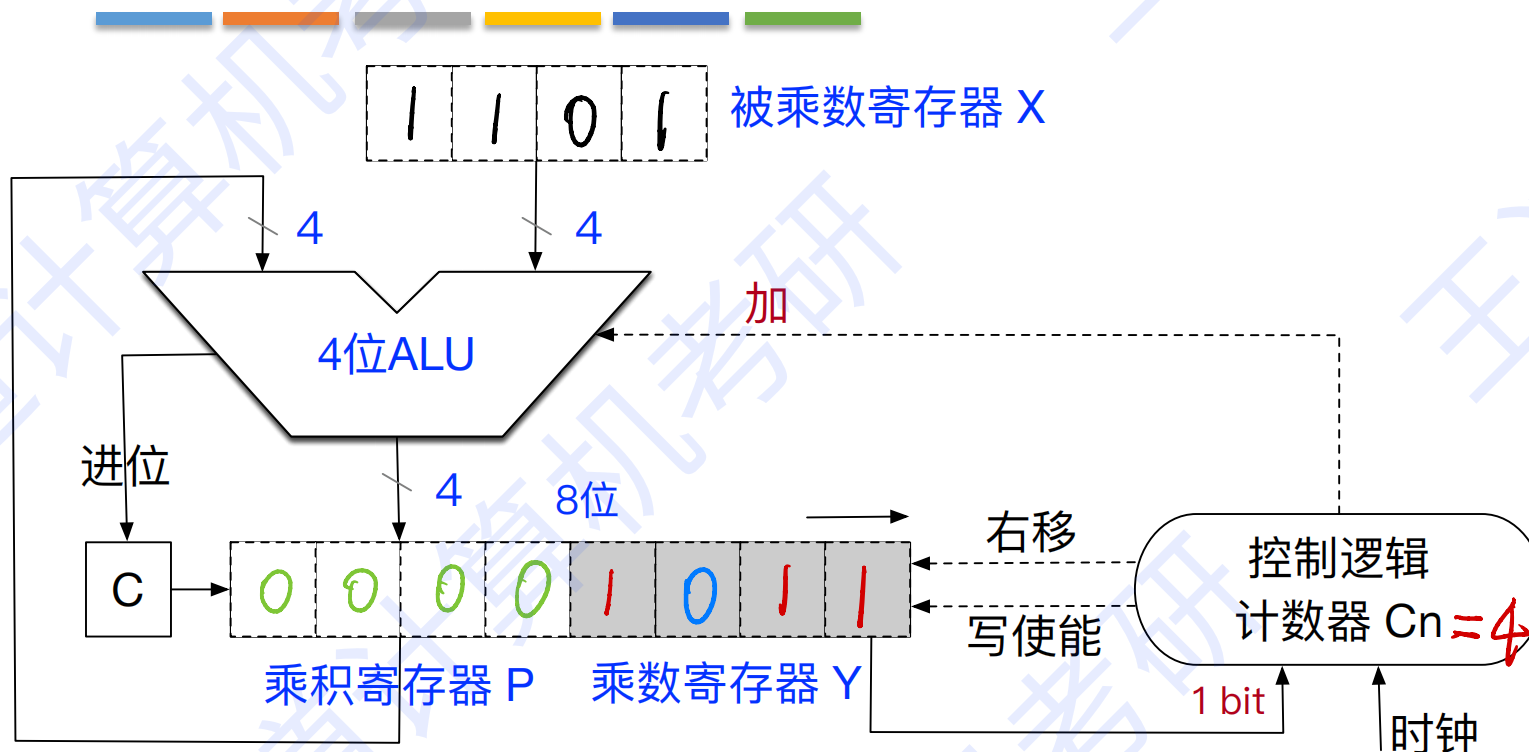


经观察可知：

- 两个 n bit 无符号数的乘法运算，可拆解为 n 轮加法运算。
- 根据乘数的各个bit，决定每一轮加法运算 “+被乘数” 或 “+全0”
- 注意每一轮加法运算需要与上一轮加法运算的结果 “错位相加”

无符号整数乘法原理：以 4bit 无符号整数为例

$$\begin{array}{r} 1101 \\ \times 1011 \\ \hline 0000 \quad \text{部分积 } P_0 \\ + 1101 \\ \hline 1101 \quad \text{部分积 } P_1 \\ + \underset{1}{1}101 \\ \hline 100111 \quad \text{部分积 } P_2 \\ + 0000 \\ \hline 100111 \quad \text{部分积 } P_3 \\ + \underset{1}{1}101 \\ \hline 10001111 \quad \text{最终乘积 } P_4 \end{array}$$



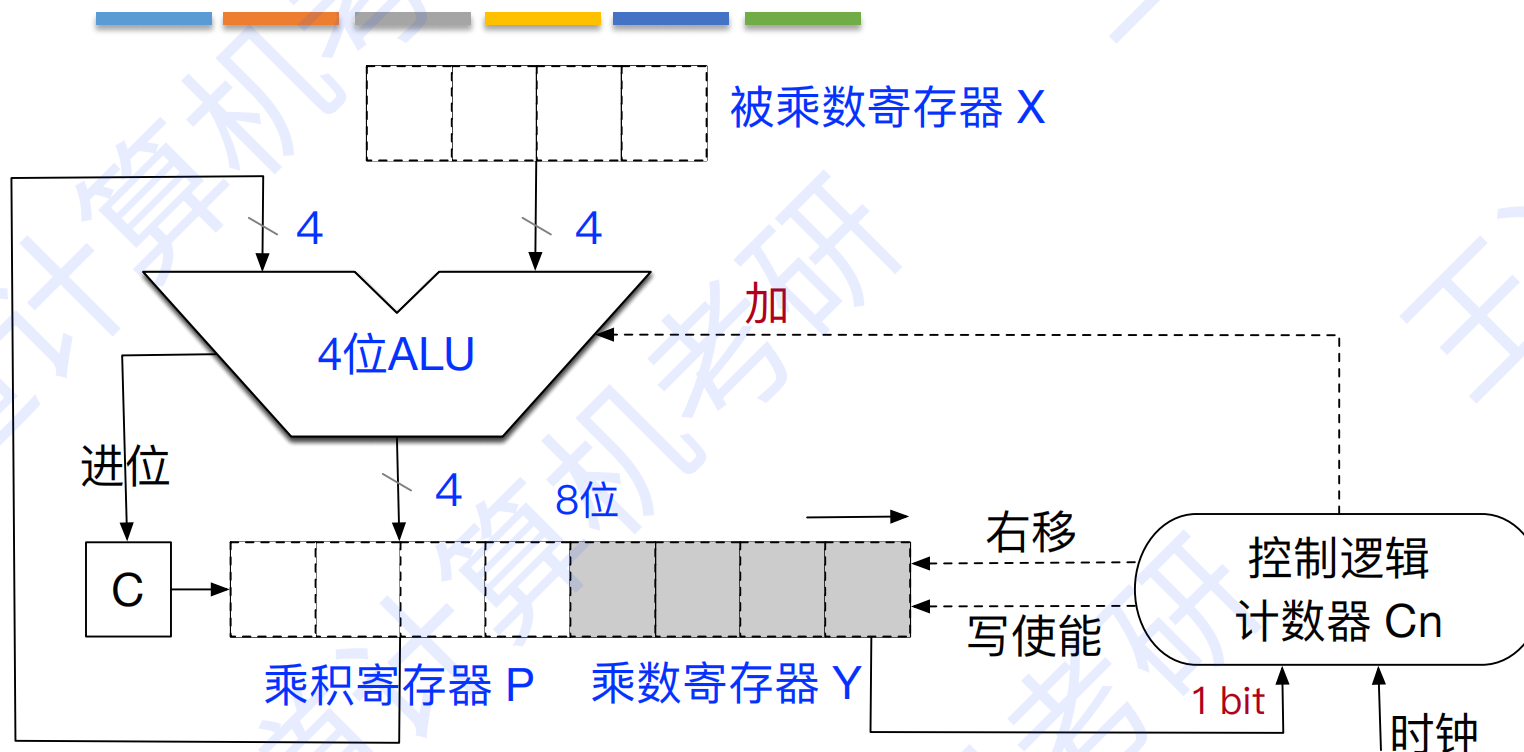
开始:

1. 将被乘数、乘数 分别放入寄存器X、Y
2. 乘积寄存器P 置为0
3. 计数器 C_n 的初始值置为 n (n 为乘数的位数)

特殊情况：当被乘数、乘数中有一个为全0时，结果直接得0，不需要再进行后续的运算步骤

无符号整数乘法原理：以 4bit 无符号整数为例

$$\begin{array}{r} 1101 \\ \times 1011 \\ \hline 0000 \quad \text{部分积 } P_0 \\ + 1101 \\ \hline 1101 \quad \text{部分积 } P_1 \\ + \underset{1}{1}101 \\ \hline 100111 \quad \text{部分积 } P_2 \\ + 0000 \\ \hline 100111 \quad \text{部分积 } P_3 \\ + \underset{1}{1}101 \\ \hline 10001111 \quad \text{最终乘积 } P_4 \end{array}$$

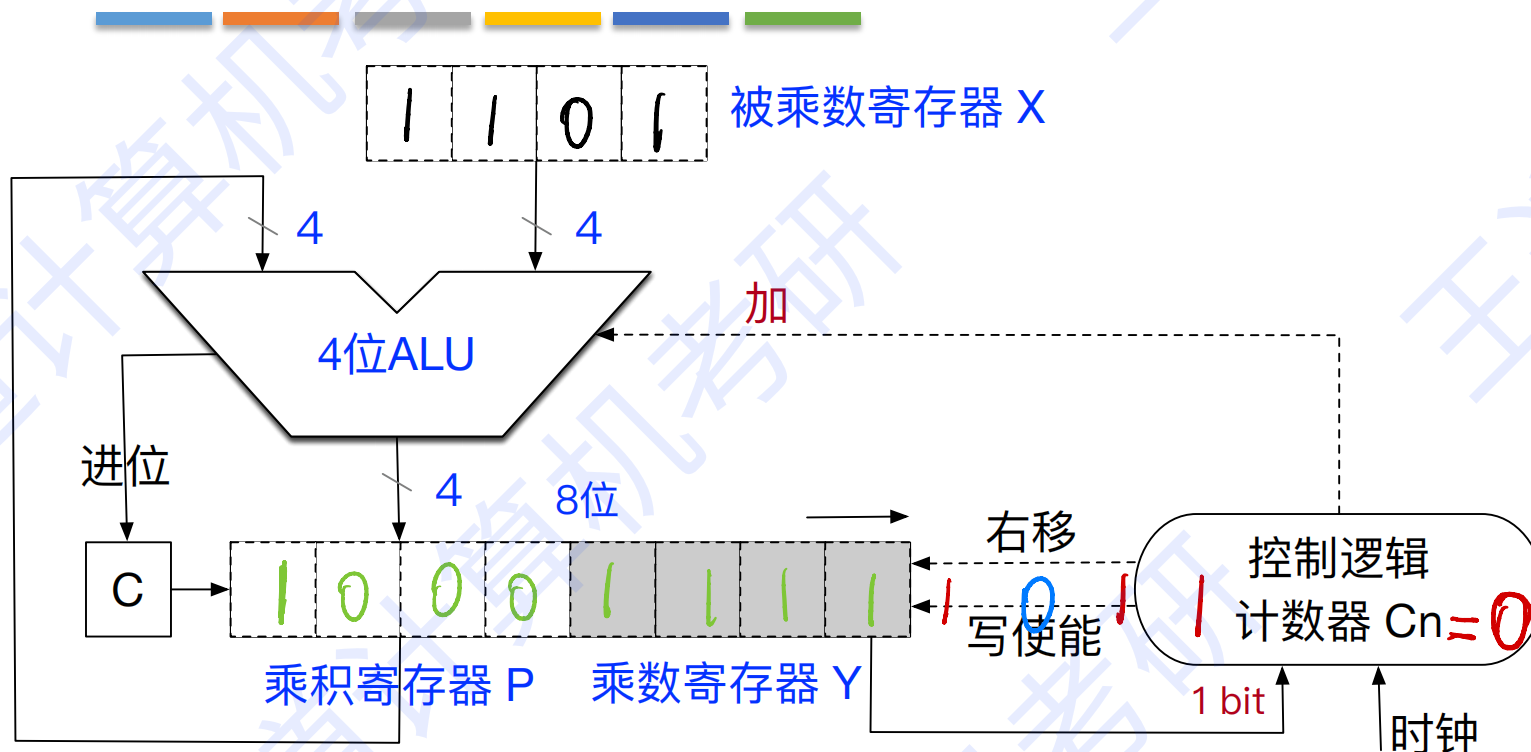


重复 n 轮加法、移位运算，直到计数器 $C_n = 0$ ：

1. 将**乘数寄存器Y的最低位**，送入“控制逻辑”进行判断
2. ①若Y的最低位为**1**，则执行**加法**，运算结果写回P（注意加法产生的进位需保存至进位触发器C）；②若Y的最低位为**0**，则**什么也不做**。
3. 将【C, P, Y】视为整体，**逻辑右移一位**
4. **计数器C_n减1**

无符号整数乘法原理：以 4bit 无符号整数为例

$$\begin{array}{r} 1101 \\ \times 1011 \\ \hline 0000 \quad \text{部分积 } P_0 \\ + 1101 \\ \hline 1101 \quad \text{部分积 } P_1 \\ + \underset{1}{1}101 \\ \hline 100111 \quad \text{部分积 } P_2 \\ + 0000 \\ \hline 100111 \quad \text{部分积 } P_3 \\ + \underset{1}{1}101 \\ \hline 10001111 \quad \text{最终乘积 } P_4 \end{array}$$



结束 (计数器 $C_n = 0$) :

- ① 乘法运算的结果用 $2n$ 位暂存 (寄存器P、寄存器Y)
- ② 在很多计算机架构中, 通常仅保留低 n 位作为乘积结果 (因此, 运算结果可能发生溢出)

无符号整数乘法原理：溢出判断、溢出处理

例1:

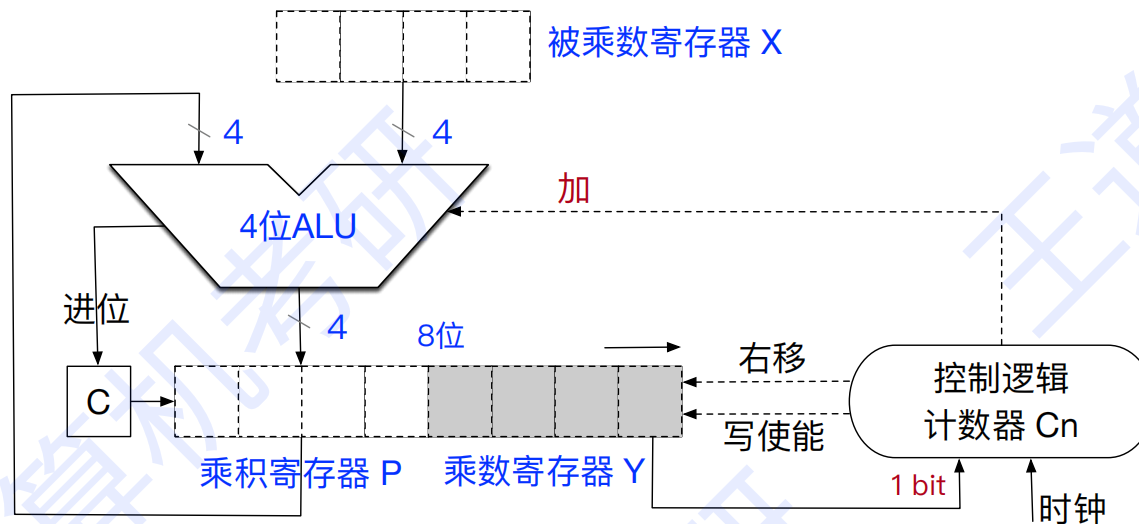
$$1101 \times 1011 = 1000\ 1111$$

$13 \times 11 = 143$ ，超出4bit无符号数表示范围

例2:

$$0100 \times 0011 = 0000\ 1100$$

$4 \times 3 = 12$ ，未超出4bit无符号数表示范围



无符号整数乘法的**溢出判断**:

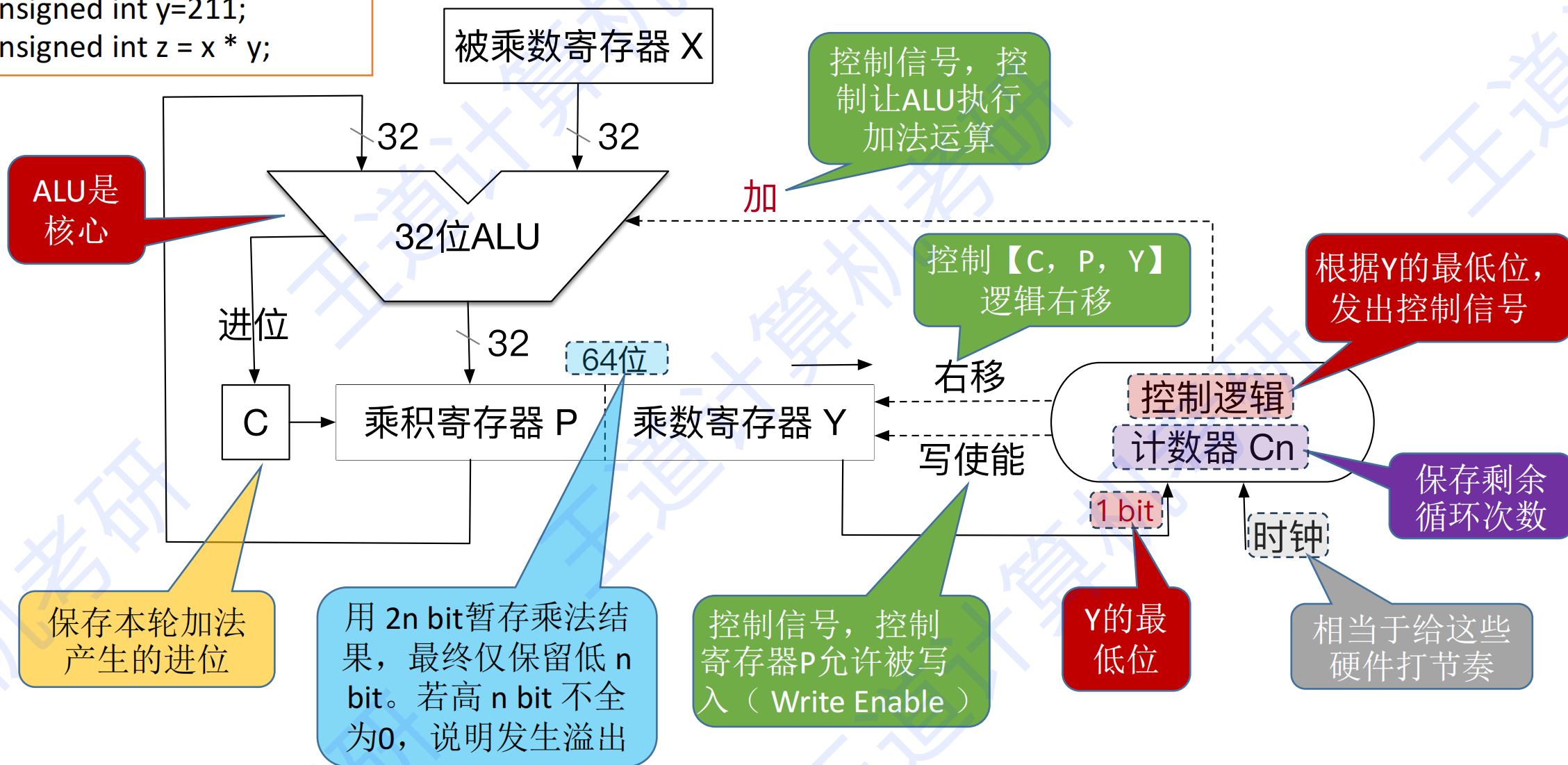
- 两个 n bit **无符号整数**相乘，运算**结果用 $2n$ bit 暂存**。通常仅保留低 n 位作为乘法运算结果。若**高 n 位不全为0**，说明**发生溢出**，此时可将**OF标志位（溢出标志位）置为1**

溢出的处理:

- ✓ 程序员可以选择**忽略“溢出”**，只不过这样会导致错误的运算结果
- ✓ 如果想要**处理“溢出”**这种**“异常”**，可以在**乘法指令之后**执行一条**溢出自陷指令**，（例如：x86 的 INTO 指令（**Interrupt on Overflow**））。**该指令会检查 OF标志位，若 $OF==1$ ，就执行操作系统的“异常处理程序”**

例如：可用该电路实现如下C语言程序的乘法运算
unsigned int x=985;
unsigned int y=211;
unsigned int z = x * y;

32 位无符号数乘法运算电路的结构



知识回顾与重要考点

无符号整数的乘法运算原理

本质是模拟无符号整数乘法的手算过程——“逐位相乘，错位相加”

运算过程

开始

①被乘数、乘数 分别放入寄存器X、Y；②乘积寄存器P 置为0；③计数器C_n 的初始值置为 n

进行 n 轮处理

重复 n 轮加法、移位运算，直到计数器C_n = 0：

1. 将**乘数寄存器Y的最低位**，送入“控制逻辑”进行判断
2. ①若Y的**最低位为1**，则执行**加法**，运算结果写回P（注意加法产生的进位需保存至进位触发器C）；②若Y的**最低位为0**，则**什么也不做**。
3. 将【C, P, Y】视为整体，**逻辑右移一位**
4. **计数器C_n减1**

结束

当计数器C_n=0 时，乘法运算结束。用 2n 位【P, Y】暂存乘法运算结果，但最终**仅保留低 n 位【Y】作为最终结果**

溢出判断

若丢弃的**高 n 位【P】不全为0**，说明发生**溢出**，并设置**OF标志位=1**

溢出处理

摆烂：程序员可以选择**忽略乘法溢出**，乘法结果错了也不管

不摆烂：也可以在**乘法指令之后执行一条“溢出自陷指令”**，当OF=1时会触发“异常处理程序”来处理溢出

